

Module Interface Specification for CEWS System

Lesley Wheat

April 17, 2023

1 Revision History

Date	Version	Notes
2022-03-10	0.0	Creation
2022-03-17	1.0	Version 1.0
2022-03-21	1.1	Revisions from feedback
2023-04-16	2.0	Revision based on feedback

2 Symbols, Abbreviations and Acronyms

See SRS Documentation at [GitHub](#).

Contents

1	Revision History	i
2	Symbols, Abbreviations and Acronyms	ii
3	Introduction	1
4	Notation	1
5	Module Decomposition	2
6	MIS of Control Module	3
6.1	Module	3
6.2	Uses	3
6.3	Syntax	3
6.3.1	Exported Constants	3
6.3.2	Exported Access Programs	3
6.4	Semantics	3
6.4.1	State Variables	3
6.4.2	Environment Variables	3
6.4.3	Assumptions	3
6.4.4	Access Routine Semantics	3
6.4.5	Local Functions	4
7	MIS of Input Module	5
7.1	Module	5
7.2	Uses	5
7.3	Syntax	5
7.3.1	Exported Constants	5
7.3.2	Exported Access Programs	5
7.4	Semantics	5
7.4.1	State Variables	5
7.4.2	Environment Variables	5
7.4.3	Assumptions	5
7.4.4	Access Routine Semantics	5
7.4.5	Local Functions	6
8	MIS of Output Module	7
8.1	Module	7
8.2	Uses	7
8.3	Syntax	7
8.3.1	Exported Constants	7
8.3.2	Exported Access Programs	7

8.4	Semantics	7
8.4.1	State Variables	7
8.4.2	Environment Variables	7
8.4.3	Assumptions	7
8.4.4	Access Routine Semantics	7
8.4.5	Local Functions	8
9	MIS of Calculation Module	9
9.1	Module	9
9.2	Uses	9
9.3	Syntax	9
9.3.1	Exported Constants	9
9.3.2	Exported Access Programs	9
9.4	Semantics	9
9.4.1	State Variables	9
9.4.2	Environment Variables	9
9.4.3	Assumptions	9
9.4.4	Access Routine Semantics	9
9.4.5	Local Functions	10
10	MIS of Graph Module	11
10.1	Module	11
10.2	Uses	11
10.3	Syntax	11
10.3.1	Exported Constants	11
10.3.2	Exported Access Programs	11
10.4	Semantics	11
10.4.1	State Variables	11
10.4.2	Environment Variables	11
10.4.3	Assumptions	11
10.4.4	Access Routine Semantics	11
10.4.5	Local Functions	12
	References	13

3 Introduction

The following document details the Module Interface Specifications for CEWS System. This program implements the Cut Edge Weight Statistic measurement, proposed by Zighed et al. (1). Complementary documents include the System Requirement Specifications and Module Guide. The full documentation and implementation can be found at [GitHub](#).

4 Notation

The base structure of the MIS for modules comes from (2), with the addition that template modules have been adapted from (3) and the addition of descriptions for readability. Note that the symbol $:=$ is used for assignment.

The following table summarizes the primitive data types used by CEWS System.

Data Type	Notation	Description
natural number	$\mathbb{N}$	a number without a fractional component in $[1, \infty)$
real	$\mathbb{R}$	any number in $(-\infty, \infty)$
matrix	$\mathbb{M}_{n \times m}$	a matrix of real numbers of size n and m

The specification of CEWS System uses some derived data types: sequences, strings, and tuples. Sequences are lists filled with elements of the same data type. Strings are sequences of characters. Tuples contain a list of values, potentially of different types. In addition, CEWS System uses functions, which are defined by the data types of their inputs and outputs. Local functions are described by giving their type signature followed by their specification.

5 Module Decomposition

The following table is taken directly from the Module Guide document for this project.

Level 1	Level 2
Hardware-Hiding Module	
Behaviour-Hiding Module	Control Module Input Module Output Module Calculation Module Graph Module
Software Decision Module	

Table 1: Module Hierarchy

6 MIS of Control Module

6.1 Module

CEWS

6.2 Uses

Input Module 7, Output Module 8, Calculation Module 9

6.3 Syntax

6.3.1 Exported Constants

None.

6.3.2 Exported Access Programs

Name	In	Out	Exceptions
cutEdgeWeight	inData: $\mathbb{M}_{n \times m}$, inLabels: $\mathbb{M}_{n \times 1}$	$J_N: \mathbb{R}$	-

6.4 Semantics

6.4.1 State Variables

None.

6.4.2 Environment Variables

None.

6.4.3 Assumptions

- The inputs are valid.

6.4.4 Access Routine Semantics

cutEdgeWeight(inData, inLabels):

- Description: Controls the information flow.
 - Pass input to Input Module 7.
 - Pass result to Calculation Module 9, receive J_N .
 - Pass J_N to Output Module 8 for verification.

- Transition: None.
- Output: $out :=$
 - J_N (computed by calculation module [9](#)).
- Exception: None

6.4.5 Local Functions

None.

7 MIS of Input Module

7.1 Module

inputModule

7.2 Uses

None.

7.3 Syntax

7.3.1 Exported Constants

None.

7.3.2 Exported Access Programs

Name	In	Out	Exceptions
processInput	inData: $\mathbb{M}_{n \times m}$, inLabels: $\mathbb{M}_{n \times 1}$	data: $\mathbb{M}_{n \times m}$, labels: $\mathbb{M}_{n \times 1}$	typeError valueError

7.4 Semantics

7.4.1 State Variables

None.

7.4.2 Environment Variables

None.

7.4.3 Assumptions

- The input data has been formatted correctly with respect to association and orientation (for example, that `inData[i]` is the data point associated with `inLabels[i]`).

7.4.4 Access Routine Semantics

`processInput(inData, inLabels):`

- Description: Verify and transform input if applicable.
 - Check if input is a correct data type
 - Check input meets constraints
 - Convert information to internal data programming object

- Transition: None
- Output: *out* :=
 - data: the inputted data converted to the correct internal data storage type (from matrix to Python Numpy matrix) for use by the Calculation Module [9](#)
 - labels: the inputted labels converted to the correct internal data storage type (from matrix to Python Numpy matrix) for use by the Calculation Module [9](#)
- Exceptions: *exc* :=
 - `TypeError`:
 - * `inData` or `inLabels` is not an accepted data object (for example, a string not a matrix).
 - * `inData` or `inLabels` contain non-numeric values.
 - * Different rows of `inData` have a different m value.
 - * Any index in `inData` or `inLabels` is empty or null.
 - `ValueError`:
 - * `inData` or `inLabels` do not have the same value of n .
 - * There is only one class label used (a minimum of two classes must exist in the dataset).
 - * The `inLabels` is not of size $n \times 1$.

7.4.5 Local Functions

None.

8 MIS of Output Module

8.1 Module

outputModule

8.2 Uses

None.

8.3 Syntax

8.3.1 Exported Constants

None.

8.3.2 Exported Access Programs

Name	In	Out	Exceptions
verifyOutput	$J_N: \mathbb{R}$	-	resultOutOfBounds

8.4 Semantics

8.4.1 State Variables

None.

8.4.2 Environment Variables

None.

8.4.3 Assumptions

None.

8.4.4 Access Routine Semantics

verifyOutput(J_N):

- Description: Check calculation value against constraints.
- Transition: None.
- Output: None.
- Exception: $exc :=$
 - resultOutOfBounds: If $J_N < 0$ or $J_N > 1$.

8.4.5 Local Functions

None.

9 MIS of Calculation Module

9.1 Module

calculationModule

9.2 Uses

Graph Module [10](#)

9.3 Syntax

9.3.1 Exported Constants

None.

9.3.2 Exported Access Programs

Name	In	Out	Exceptions
calcCEWS	data: $\mathbb{M}_{n \times m}$ labels: $\mathbb{M}_{n \times 1}$	$J_N: \mathbb{R}$	-

9.4 Semantics

9.4.1 State Variables

None.

9.4.2 Environment Variables

None.

9.4.3 Assumptions

- Input variables have been processed through Input Module [7](#).

9.4.4 Access Routine Semantics

calcCEWS(data, labels):

- Description: Perform the algorithm from [\(1\)](#).
- Transition: None.
- Output: $out :=$
 - The Cut Edge Weight Statistic: $J_N := \frac{J}{I+J}$ where:

- * $J = \sum_{r=1}^{k-1} \sum_{s=r+1}^k J_{r,s}; 0 \leq J \leq n^2$
- * $J_{r,s} = \sum_{i=1}^n \sum_{j=1}^n V_{ij} I(\text{labels}_i = r \ \& \ \text{labels}_j = s)$
- * $I = \sum_{r=1}^k I_r, 0 \leq I \leq n^2$
- * $I_r = \sum_{i=1}^{n-1} \sum_{j=i+1}^n V_{ij} I(\text{labels}_i = r \ \& \ \text{labels}_j = r).$
- * V is obtained from the Graph Module [10](#) for the data input.

- Exceptions: None.

9.4.5 Local Functions

None.

10 MIS of Graph Module

10.1 Module

graphModule

10.2 Uses

None.

10.3 Syntax

10.3.1 Exported Constants

None.

10.3.2 Exported Access Programs

Name	In	Out	Exceptions
relativeNeighbourGraph	data: $\mathbb{M}_{n \times m}$	$V: \mathbb{M}_{n \times n}$	-

10.4 Semantics

10.4.1 State Variables

None.

10.4.2 Environment Variables

None.

10.4.3 Assumptions

- data has been processed by Input Module [7](#).

10.4.4 Access Routine Semantics

relativeNeighbourGraph(data):

- Description: Construct a relative neighbour graph.
- Transition: None.
- Output: $out :=$

- Connection matrix $V := (v_{ij}), i = 1, 2, \dots, n; j = 1, 2, \dots, n;$
 where $v_{ij} := \text{checkEdge_RNG}(\text{distanceMatrix}, i, j)$ given:
 $\text{distanceMatrix}_{ij} = d(\text{data}_i, \text{data}_j)$ where $d(a, b)$ is Euclidean Distance.

- Exceptions: None.

10.4.5 Local Functions

$\text{checkEdge_RNG}(\text{distanceMatrix}, a, b)$:

- Description: Checks if an edge exists between points i and j .
- Inputs:
 - distanceMatrix: a distance matrix, $\mathbb{M}$, size of $n \times n$
 - a : $\mathbb{N}, 1 \leq a \leq n$
 - b : $\mathbb{N}, 1 \leq b \leq n$
- Transition: None.
- Output: $\text{out} :=$
 - Edge existence: $E := I([\text{distanceMatrix}_{ab} \leq \max(\text{distanceMatrix}_{ac}, \text{distanceMatrix}_{bc}) \forall c \text{ where } 1 \leq c \leq n, c \neq a, c \neq b])$.
- Exceptions: None.

References

- [1] D. A. Zighed, S. Lallich, and F. Muhlenbach, “A statistical approach to class separability,” *Applied Stochastic Models in Business and Industry*, vol. 21, no. 2, pp. 187–197, Mar. 2005. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1002/asmb.532>
- [2] D. M. Hoffman and P. A. Strooper, *Software Design, Automated Testing, and Maintenance: A Practical Approach*. New York, NY, USA: International Thomson Computer Press, 1995. [Online]. Available: <http://citeseer.ist.psu.edu/428727.html>
- [3] C. Ghezzi, M. Jazayeri, and D. Mandrioli, *Fundamentals of Software Engineering*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.